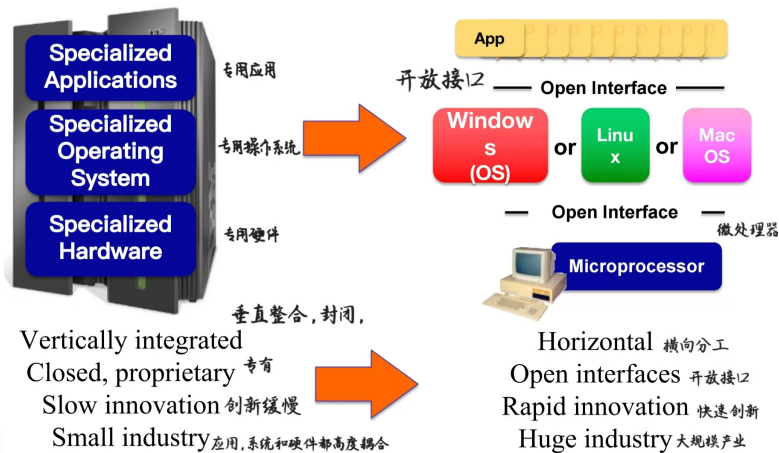
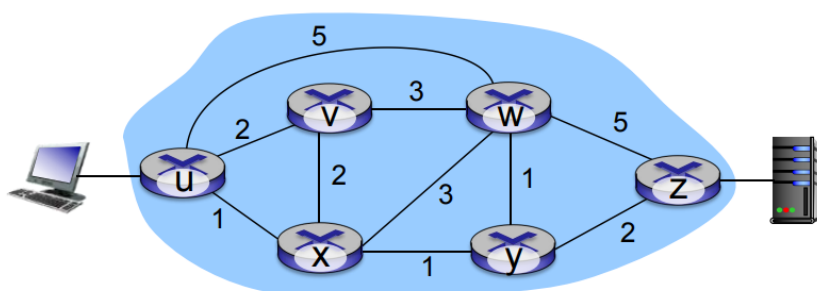


lecture 13

- 为什么要区分域内与域间路由？
 - policy（策略）
 - 域间（inter-AS）：网络管理员希望控制本网络流量如何被路由、哪些流量可以通过其网络。
 - 域内（intra-AS）：属于同一管理员/组织，所以无需做复杂的策略决定。
 - performance（性能）
 - 域间：策略可能优先于性能考虑。
 - 域内：可以聚焦于性能优化。
 - scale（可扩展性）
 - 分层路由能节省路由表大小并减少更新流量。
- The SDN control plane
 - Software defined networking (SDN) 软件定义网络
 - 为什么采用逻辑集中化的控制平面？
 - 更容易的网络管理：避免路由器配置错误，提供更大的流量调度灵活性。
 - 说明：集中控制可以统一下发策略、检测并减少配置不一致，从而降低人为出错并能灵活地按策略调整流量路径。
 - 基于表的转发（参考 OpenFlow API）允许“编程”路由器。
 - 说明：SDN 用流表（match→action）来实现转发，控制器可以下发表项，像编程一样控制路由器的转发行为。
 - 集中式“编程”更容易：在中心计算表项然后分发。
 - 说明：控制器全局视图下可以全网优化计算路由/策略表，再批量下发到交换/路由设备，避免每台设备各自运行复杂算法。
 - 分布式“编程”更困难：需通过在每台路由器上实现分布式算法来计算表项（协议）。
 - 说明：传统分布式路由需要每台设备独立运行收敛算法（如 OSPF、BGP），实现复杂且收敛时间/一致性难以控制。
 - 控制平面的实现是开放的（非专有的）。
 - 说明：SDN 倡导开放接口（如 OpenFlow）和可编程控制器，促使多厂商互操作和快速创新，而不是厂商锁定的专有堆栈。
 - 类比：从大型机到个人电脑的演进



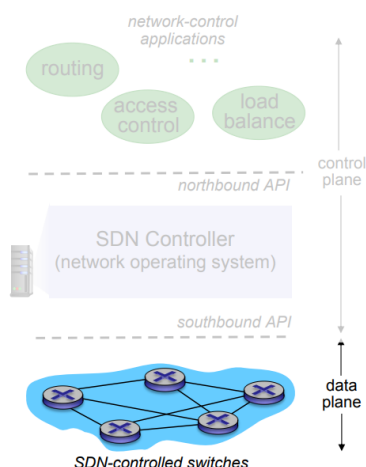
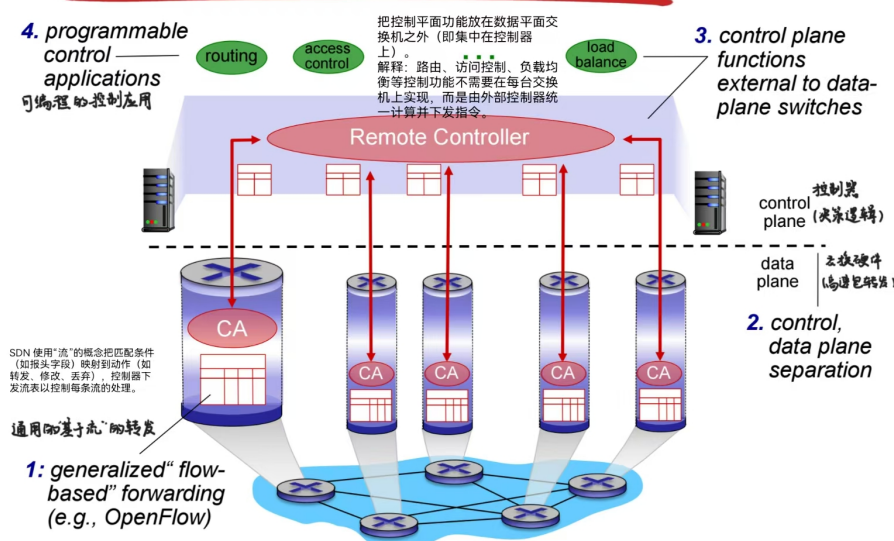
- Traffic engineering: difficult traditional routing



- 问题: 如果网络运营者希望 $u \rightarrow z$ 的流量走 $uvwz$, 而 $x \rightarrow z$ 的流量走 $xywz$?
 - 回答: 需要设置链路权重, 使得路由算法据此计算出期望路径 (或者需要一种新的路由算法)。
 - 说明: 在基于最短路径的协议 (如 OSPF) 中, 路由决策依据链路权重, 运营者可尝试通过调整权重来诱导路径, 但这在多流量场景下很难精确控制。
 - 但链路权重不能直接任意设为某个数值 (或权重的设置并不能直接映射到所想要的流量分布)。
- 问题: 若运营者希望把 $u \rightarrow z$ 流量拆分到 $uvwz$ 与 $uxyz$ 两条路径上 (负载均衡) ?
 - 回答: 无法做到 (或需要一种新的路由算法)。
 - 说明: 传统基于最短路径的路由器不能按流量百分比分流同源流量到多条独立路径 (除非使用 ECMP 等受限方法), 要实现精确负载均衡通常需要集中控制或 MPLS/源路由等技术。
- 问题: 如果节点 W 想把蓝色流量和红色流量分别走不同路径, 该怎么办?
 - 回答: 做不到 (在基于目的地址的转发, 以及基于链路状态/距离向量的路由下无法实现)
 - 说明: 基于目的地址的转发只看目标 IP (或目标前缀), 不区分不同流 (例如来自不同源或不同应用) 的流量; 如果蓝色和红色流量去往相同目的地, 路由器就只能选一个“到该目的地”的下一跳, 不能对它们做不同转发。
 - 传统的路由算法 (链路状态 LS, 距离向量 DV) 计算的是“每个目的地的最佳路径”, 所以自然也不能对同一目的地的不同流量选用多条不同路径。

- 要实现按流区分的路径，需要超出目的地址转发的机制，比如策略路由（PBR）、基于源地址或五元组的路由、MPLS/Traffic Engineering（RSVP-TE、Segment Routing）、或用 SDN 直接下发流表规则。另一种简单办法是把流量映射到不同的目的前缀（例如 NAT 或不同的目标地址），让常规路由表产生不同下一跳。

Software defined networking (SDN)

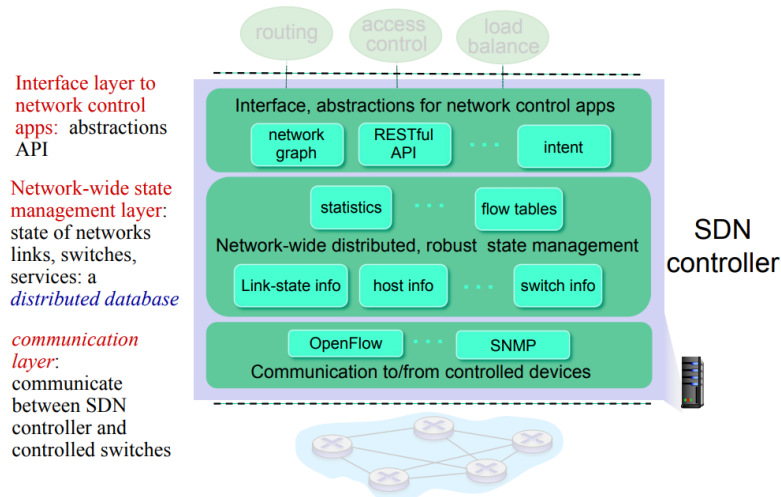


• Data plane switches 数据平面交换机

- 快速、简单的商品化交换机，在硬件中实现通用的数据平面转发。
 - SDN 要求数据面设备尽量简单、支持高速转发，使用专用硬件（ASIC）来执行表查找和转发动作，从而保证性能。
- 控制器负责把“流表条目”（匹配 + 动作）计算好后，使用南向协议把这些表项写入交换机，使交换机按控制器设定的策略转发流量。
- 用于表驱动交换机控制的 API（例如 OpenFlow）——定义哪些是可控的、哪些不可控。
 - 北/南向接口规范化了控制器与交换机交互的能力范围（例如能下发哪些匹配项、能读取哪些统计），OpenFlow 是典型的南向协议示例。
- 交换机与控制器通过可靠的控制通道（TCP 上的 OpenFlow 或其它协议）交换消息：

- 下发流表、上报出错/未命中流的信息、收集统计等。
- **SDN controller (network OS) SDN 控制器（网络操作系统）：**
 - 控制器在 SDN 中相当于“网络的操作系统”，为上层应用提供抽象与能力，为下层交换机提供统一管理。
 - 维护网络状态信息。
 - 控制器收集并保存拓扑信息、链路/端口状态、流量统计、设备能力等，是决策的全局数据源。
 - 通过北向 API 与上层网络控制应用交互。
 - 路由规划、访问控制、流量工程等应用使用北向接口调用控制器服务（例如请求安装某条路径或查询拓扑），实现业务与策略逻辑。
 - 通过南向 API 与下层网络交换机交互。
 - 控制器通过南向协议（如 OpenFlow、gNMI、NETCONF）下发流表、接收事件与统计数据，完成对交换机的控制与监控。
 - （控制器）以分布式系统方式实现，以获得性能、可扩展性、容错性与鲁棒性。
 - 尽管逻辑上是“集中”，实际部署通常是多个控制器实例分布式运行（leader/replica 或分区式），以避免单点故障并支撑大规模网络与高可用需求。
 - SDN 控制器的组成部分：

Components of SDN controller



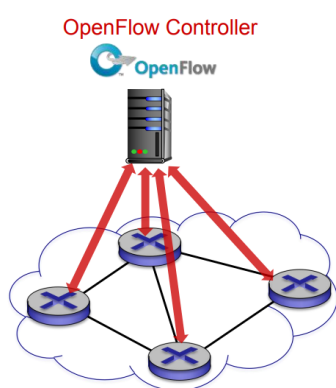
- 面向网络控制应用的接口层：抽象与 API。
 - 说明：这一层为上层应用提供抽象（如网络图、意图接口、RESTful API），屏蔽底层设备细节，使应用可以基于全网抽象编程。
- 全网状态管理层：维护网络、链路、交换机、服务的状态；是一个分布式数据库。
 - 说明：控制器需要保持关于拓扑、链路利用率、主机位置、流统计、流表状态等全局视图，这通常以分布式状态存储实现以保证扩展性和容错。
- 通信层：负责 SDN 控制器与被控交换机之间的通信。

- 说明：该层实现南向协议（如 OpenFlow、gNMI、NETCONF），用于下发表项、接收事件、收集统计数据等，是控制器与数据平面设备的桥梁。

- **network-control apps: 网络控制应用**

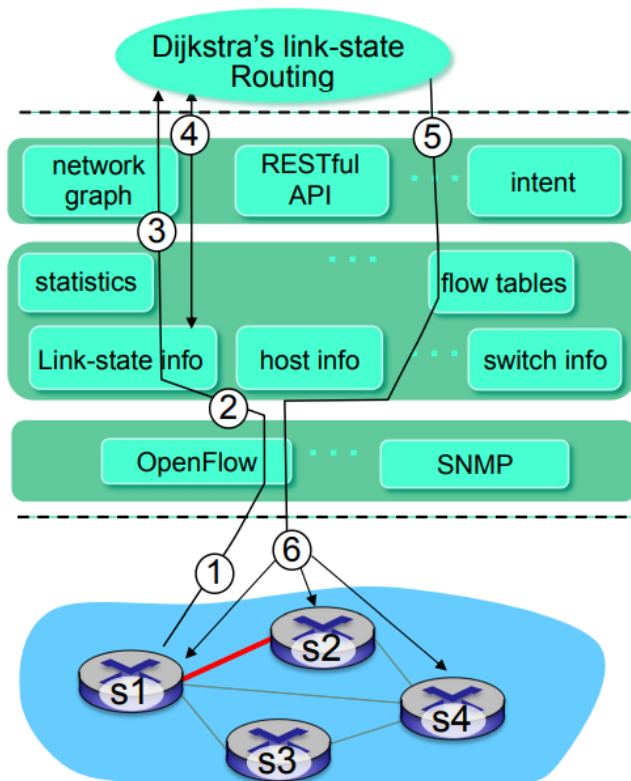
- 指运行在控制器之上的程序模块，负责实现具体的网络功能或策略
- 使用由 SDN 控制器提供的底层服务与 API 来实现控制功能。
 - 说明：控制应用是决策和策略实现的主体（例如决定流应走哪条路径），它们不直接操作设备硬件，而通过控制器的北向/南向接口调用控制器能力或下发命令到交换机。
- 解耦/拆分：这些应用可以由第三方提供，独立于路由器厂商或 SDN 控制器厂商。
 - 说明：SDN 的开放架构允许不同厂商或开发者编写控制应用（例如专业负载均衡、流量分析、安全策略），从而形成生态，而不是把所有功能绑定在单一硬件或厂商堆栈中。

- **OpenFlow 协议**



- 本页简述 OpenFlow 作为最早且典型的 SDN 南向协议，用于控制器与交换机之间的消息交换
- OpenFlow 定义了控制器如何与交换设备建立会话、下发流表、接收未命中/上报事件等交互。
- 使用 TCP 交换消息。
 - OpenFlow 运行在 TCP 之上（通常端口 6653），以可靠传输方式保证消息的顺序与到达；控制平面通信因此具备可靠语义。
- OpenFlow 消息类别：
 - 控制器 → 交换机
 - 下发命令（配置、流表）
 - configure（配置）：控制器查询或设置交换机的配置参数。
 - 说明：例如设置交换机的端口属性、缓冲阈值或启用/禁用某些功能，属于设备级的配置动作。
 - modify-state（修改状态）：在 OpenFlow 流表中添加、删除、修改流条目
 - 说明：这是控制器下发流表的核心操作（如给某类流安装转发表项或撤销旧规则），直接决定数据平面的转发行为。

- Read-state（读取状态）：从交换机的流表和端口收集统计与计数器值
 - 说明：控制器会周期性或按需读取统计（字节/包计数、端口利用率等），用于流量监控、决策与流量工程反馈闭环。
- packet-out：控制器可以命令交换机把某个报文从指定的接口发送出去
 - 说明：在某些场景下（如 ARP 或需要注入控制包），控制器直接下发一个“发包”动作到交换机，可在未在流表中预安装规则时触发特定数据平面行为。
- 交换机 → 控制器
 - 上报事件/统计
 - packet-in：把某个数据包（及其控制上下文）发给控制器。参见控制器的 packet-out 消息
 - 解释：当交换机收到一个未命中的包或按策略需要控制器决策时，会把该报文（或报文首部/摘样）通过 packet-in 发给控制器，请求如何处理（例如安装流表或直接下发该包）。控制器可用 packet-out 指令让交换机发送包或下发相应流表。
 - flow-removed：交换机上某条流表条目被删除（或超时移除）
 - 解释：当交换机上的某条流规则因超时、被显式删除或因资源回收而移除时，会通知控制器。控制器可据此更新统计、重新下发条目或触发应用逻辑。
 - port status（端口状态）：通知控制器端口状态发生变化
 - 解释：例如链路故障、端口上/下事件、速率变化等，交换机通过 port-status 通知控制器，使控制器能更新拓扑/链路状态并触发相应的控制应用（重路由、流表更新等）。
- SDN：控制/数据平面交互示例



- 图中用编号 1-6 表示一次链路故障触发到控制器计算并下发新流表的完整交互流程
 - 步骤 1
 - 数据平面设备（交换机 s1）检测到本地链路或端口状态变化后，使用 OpenFlow 的 port-status 消息将该事件上报给控制器（南向消息），这是控制器感知网络变化的起点。
 - 步骤 2
 - SDN 控制器接收 OpenFlow 消息，把上报的端口/链路状态写入其全局状态库（分布式数据库中的 link-state info），用于后续计算与应用调用，并更新链路状态信息。
 - 步骤 3
 - Dijkstra 链路状态路由算法应用事先注册为“当链路状态变化时被调用”的回调程序。现在控制器调用它。
 - 控制器上运行的控制应用（例如链路状态路由应用）在检测到状态变化后被触发，开始基于最新的全局链路信息重新计算路径。
 - 步骤 4
 - Dijkstra 算法访问控制器中的网络图信息与链路状态信息，计算新的路由。
 - 因为控制器有全网视图，算法可以直接在控制器端运行 Dijkstra（或其它优化算法）来得到新的最短路径集或流表更新集合。
 - 步骤 5
 - 链路状态路由应用与控制器中的“流表计算”组件交互，后者计算需要安装的新流表（条目）

- 路由计算得出路径后，需把高层路径映射为具体的交换机流表（匹配条件 + 动作），这个映射由控制器的流表计算模块负责生成具体条目。
- 步骤 6
 - 控制器使用 OpenFlow 把计算好的流表下发到相关交换机（modify-state），实现数据平面的即时调整，从而完成一次端到端的故障检测→重计算→下发的闭环。
- 与早期“每路由器独自控制”方案相比，有两个重要差异：
 - 差异一：Dijkstra 算法作为一个独立的应用执行，在数据包交换机之外运行。
 - 解释：传统链路状态中每台路由器都运行 Dijkstra，SDN 把它作为控制器上的应用集中运行，从而避免在每台设备重复计算并能利用全网一致视图与额外信息（统计、策略）优化路径。
 - 差异二：数据包交换机把链路更新发送给 SDN 控制器，而不是相互之间发送。
 - 解释：传统分布式协议要求路由器间互相洪泛更新（逐跳传播）；在 SDN 中，交换机只需向控制器上报，控制器负责统一传播/处理，减少了交换机间的洪泛与分散收敛问题。

• ICMP：网际控制报文协议

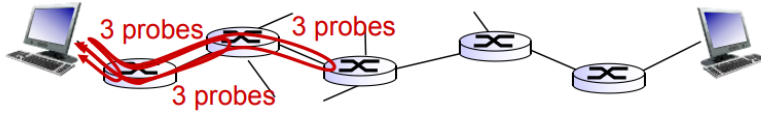
- ICMP 是 IP 之上的控制/诊断协议，用于主机和路由器交换网络层级别的信息（错误、诊断等）

Type	Code	description
0	0	echo reply (ping)
3	0	dest. network unreachable
3	1	dest host unreachable
3	2	dest protocol unreachable
3	3	dest port unreachable
3	6	dest network unknown
3	7	dest host unknown
4	0	source quench (congestion control - not used)
8	0	echo request (ping)
9	0	route advertisement
10	0	router discovery
11	0	TTL expired
12	0	bad IP header

- 由主机和路由器用于通信网络层级别的信息。
- 说明：ICMP 并非为终端应用（如 HTTP）设计，而用于传递诸如不可达、超时、路由问题等网络事件。
 - 错误报告：主机/网络/端口/协议不可达。
 - 说明：当数据包在传输过程中发生错误（例如目的端口不存在或中间路由不可达），会产生相应的 ICMP 错误消息通知源主机。
 - 回显请求/回显应答（用于 ping）。
- 网络层“位于”IP 之上：ICMP 报文的协议字段在 IP 头里（protocol=1），ICMP 报文作为 IP 的有效载荷被转发。

- ICMP 报文结构：类型 + 代码 + 导致错误的 IP 数据报头和前 8 字节（通常为原始传输层头部）。

- Traceroute and ICMP



- 常规 traceroute 会为每个跳发送 3 个探测并显示三组 RTT，以便观察波动与丢包。
- 经典的 traceroute（Unix 版）源主机向目的端口（通常为高号、不太可能被占用端口）发送一系列 UDP 段（探测包），逐次增加 TTL。
 - 第一批的 TTL = 1
 - 第二批的 TTL = 2，依此类推
 - 使用不太可能被占用的端口号
 - 说明：用递增 TTL 的方式让中间每一跳分别触发 TTL 超时并返回 ICMP，目的使用“unlikely port”以便当探测到目标时目标会返回“端口不可达”而告知终止。
- 当第 n 批的报文到达第 n 个路由器时：
 - 路由器丢弃该数据报，并向源发送 ICMP 消息（类型 11，代码 0）。
 - 说明：这是 traceroute 的核心机制：TTL 到 0 时路由器返回 ICMP TTL exceeded，从而让源知道第 n 跳的路由器地址与 RTT。
 - ICMP 报文包含路由器的名称和 IP 地址（通常为报文来源 IP，反向 DNS 可能被解析为名称）。
 - 说明：源可以根据这些信息构建跳数列表并测量往返时延。
- 当 ICMP 报文到达时，源主机记录往返时延（RTT）。
 - 说明：通过对每跳发送多个探测并记录时间戳来估计该跳的延迟及丢包率。
- 停止条件：
 - UDP 段最终到达目的主机（说明已到达目标）。
 - 目的主机返回 ICMP “端口不可达”（类型 3，代码 3）。
 - 源主机停止发送探测。
 - 说明：当探测抵达目标主机且目标返回端口不可达时，traceroute 能确定已经到达并停止；某些实现用 ICMP Echo 而不是 UDP，但原理类似。